Faculty of Computers and Artificial Intelligence – Fayoum University

Parallel Computing – Computer Science – Practical Project Sheet

V1.0

Course Title	Course Team
Parallel Computing	Dr. Ayman Antar Al-Helbawy Eng. Rehab Mahmoud (rm1707@fayoum.edu.eg) Eng. Mahmoud Badry (mma18@fayoum.edu.eg)
Project Title	**Project Submission Information**
Advanced Parallel Grid Processing with MPI	• This is a team-based project. • Each student must join a team of **6** members at **maximum**, **3** at **minimum.** • Each team should send an email with the title "**[PC-2026] Team Members**" before 26-04-2026 • The project will be discussed on-site (with a demo) in the faculty and will be defended orally.
Submission Date	
Final Exam Day (TBD) • Each team will have an announced **delivery time** that will be public on our Google Classroom	
Grading and Assessment	
This project is marked out of 15 points. **Bonus**: +2 points	
Learning outcomes	
• Understanding of Parallel Programming Concepts • Advanced MPI Programming • Problem Decomposition and Parallelization • Communication Design & Optimization • Deadlock Analysis & Avoidance • Performance Optimization & Evaluation • Teamwork and Collaboration	

IT IS YOUR RESPONSIBILITY TO KEEP RECORDS OF ALL WORK SUBMITTED.

COPYING FROM EACH OTHER ISN'T ALLOWED.

Advanced Parallel Grid Processing with MPI

Each team will develop a parallel distributed processing system using MPI in C++. The system must process 2D data (grid or matrix-based) using parallel decomposition and communication between processes.

Core Requirements

1. **Multiple Parallel Algorithms**
    a. **Implement** at least **TWO** different parallel algorithms from different categories:
        i. Category A – Grid / Spatial:
            1. Heat Diffusion (Stencil)
            2. Game of Life
            3. Image Processing (Guided Filter)
        ii. Category B – Data / Computation
            1. Odd-Even Sort
            2. Prefix Sum
            3. Matrix Multiplication
    b. **Constraints**:
        i. Work correctly in parallel
        ii. Use real inter-process communication
        iii. Use **Big data** (for example, load large matrices from files before multiplying)
        iv. Allow selecting the algorithm at runtime
2. **Communication Design**
    a. Your system must implement **multiple** communication strategies:
        i. Blocking communication (`MPI_Send / MPI_Recv`)
        ii. Non-blocking communication (`MPI_Isend / MPI_Irecv`)
        iii. At least one additional pattern, such as:
            1. Ring communication
            2. Neighbor exchange
            3. Pipeline communication
            4. Collective-based communication
3. **Deadlock Handling**
    a. Identify a real deadlock scenario in your system
    b. Explain:
        i. Why it happens
        ii. When it happens
    c. Provide a corrected solution
4. **Data Distribution & Scalability**
    a. Handle any number of processes ($N \geq 2$)
    b. Support uneven data sizes
5. **Process Organization**
    a. Use MPI features such as: `MPI_Comm_split` or equivalent
    b. To organize processes into logical groups for:
        i. Parallel stages
        ii. Subtasks
        iii. Or structured communication
6. **Performance & Analysis**
    a. Run your system with different configurations:
        i. Vary number of processes
        ii. Provide simple observations:
            1. Performance behavior
            2. Communication impact

Bonus

1. Run correctly on multi-process setup
2. 2D data decomposition
3. Advanced load balancing

Submission files

1. Source Code
2. Short Documentation:
    a. Design overview
    b. Communication strategies
    c. Deadlock explanation
    d. Performance observations
    e. Demo Screenshots